

Labolatorium 8

April 29, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Implementacja modeli oświetlenia Lamberta i Phong'a dla scen złożonych z jednego oraz dwóch trójkątów przy rzutowaniu równoległym na płaszczyznę OXY.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
%matplotlib inline
```

2 Zadania

2.1 Zadanie 1.

Rasteryzuję pojedynczy trójkąt i wyznaczam jego jasność w modelu Lamberta dla dwóch położen punktowego źródła światła. Natężenie światła punktowego tłumię czynnikiem $\frac{1}{1+\alpha d^2}$, dzięki czemu obraz zależy zarówno od kąta padania, jak i od odległości od źródła.

```
[2]: IMAGE_SIZE = 256
PIXEL_SIZE = 0.1
AMBIENT_INTENSITY = 20.0
POINT_LIGHT_INTENSITY = 235.0
ATTENUATION_ALPHA = 0.001
SURFACE_REFLECTANCE = 1.0

TRIANGLE_1 = np.array([
    [9.0, 3.0, 16.0],
    [-3.0, 9.0, 6.0],
    [-3.0, -3.0, 4.0],
], dtype=np.float64)
TRIANGLE_2 = np.array([
    [9.0, 3.0, 16.0],
    [-3.0, -3.0, 4.0],
    [9.0, -6.0, 5.0],
], dtype=np.float64)
LIGHT_POSITIONS = {
```

```

    'Światło (a)': np.array([-3.0, -3.0, -10.0]),
    'Światło (b)': np.array([3.0, 3.0, 3.0]),
}
PHONG_EXPONENTS = [2, 5, 15, 30]
OBSERVER_EYE = np.array([5.0, 5.0, 3.0])

def create_coordinate_axes(image_size: int, pixel_size: float) -> tuple[np.
    ↪ ndarray, np.ndarray]:
    """Tworzy współrzędne środków pikseli dla obrazu o zadanym rozmiarze."""
    x_coordinates = (np.arange(image_size) + 0.5 - image_size / 2.0) * ↪
    ↪ pixel_size
    y_coordinates = (image_size / 2.0 - (np.arange(image_size) + 0.5)) * ↪
    ↪ pixel_size
    return x_coordinates, y_coordinates

def unit_triangle_normal(vertices: np.ndarray) -> np.ndarray:
    """Wyznacza znormalizowany wektor normalny trójkąta skierowany ku ↪
    ↪ dodatniemu Z."""
    normal = np.cross(vertices[1] - vertices[0], vertices[2] - vertices[0])
    normal = normal / np.linalg.norm(normal)
    if normal[2] < 0:
        normal = -normal
    return normal

def barycentric_inverse(projected_vertices: np.ndarray) -> np.ndarray:
    """Przygotowuje macierz odwrotną do szybkiego liczenia wag barycentrycznych.
    ↪ """
    return np.linalg.inv(np.vstack((projected_vertices.T, np.ones(3))))

def barycentric_weights(point_xy: np.ndarray, inverse_matrix: np.ndarray) -> np.
    ↪ ndarray:
    """Wyznacza współczynniki barycentryczne punktu na płaszczyźnie OXY."""
    return inverse_matrix @ np.array([point_xy[0], point_xy[1], 1.0])

def attenuation(distance: float, alpha: float = ATTENUATION_ALPHA) -> float:
    """Modeluje spadek natężenia oświetlenia wraz z odległością od źródła."""
    return 1.0 / (1.0 + alpha * distance * distance)

def lambert_intensity(point: np.ndarray, normal: np.ndarray, light_position: np.
    ↪ ndarray) -> float:

```

```

"""Oblicza jasność punktu zgodnie z modelem Lamberta."""
light_vector = light_position - point
distance = float(np.linalg.norm(light_vector))
if distance == 0.0:
    return AMBIENT_INTENSITY * SURFACE_REFLECTANCE
light_direction = light_vector / distance
diffuse = POINT_LIGHT_INTENSITY * SURFACE_REFLECTANCE * max(0.0, float(np.
↪dot(normal, light_direction)))
return AMBIENT_INTENSITY * SURFACE_REFLECTANCE + diffuse * ↵
↪attenuation(distance)

def phong_intensity(point: np.ndarray, normal: np.ndarray, light_position: np.
↪ndarray, eye_position: np.ndarray, shininess: float) -> float:
    """Oblicza jasność punktu zgodnie z modelem odbicia Phong."""
    light_vector = light_position - point
    light_distance = float(np.linalg.norm(light_vector))
    if light_distance == 0.0:
        return AMBIENT_INTENSITY * SURFACE_REFLECTANCE
    light_direction = light_vector / light_distance
    diffuse_factor = max(0.0, float(np.dot(normal, light_direction)))
    diffuse = POINT_LIGHT_INTENSITY * SURFACE_REFLECTANCE * diffuse_factor

    view_vector = eye_position - point
    view_distance = float(np.linalg.norm(view_vector))
    if view_distance == 0.0:
        specular = 0.0
    else:
        view_direction = view_vector / view_distance
        reflection_direction = 2.0 * diffuse_factor * normal - light_direction
        reflection_norm = float(np.linalg.norm(reflection_direction))
        if reflection_norm == 0.0 or diffuse_factor == 0.0:
            specular = 0.0
        else:
            reflection_direction = reflection_direction / reflection_norm
            specular = POINT_LIGHT_INTENSITY * max(0.0, float(np.
↪dot(reflection_direction, view_direction))) ** shininess

    return AMBIENT_INTENSITY * SURFACE_REFLECTANCE + (diffuse + specular) * ↵
↪attenuation(light_distance)

def render_scene(triangles: list[np.ndarray], shader, light_position: np.
↪ndarray, eye_position: np.ndarray | None = None, shininess: float | None = ↵
↪None) -> tuple[np.ndarray, np.ndarray]:

```

```

"""Rasteryzuję scenę trójkątów z buforem głębokości i zadany  modelem  
 o wietlenia."""
    image = np.zeros((IMAGE_SIZE, IMAGE_SIZE), dtype=np.float64)
    z_buffer = np.full((IMAGE_SIZE, IMAGE_SIZE), -np.inf, dtype=np.float64)
    mask = np.zeros((IMAGE_SIZE, IMAGE_SIZE), dtype=bool)
    x_coordinates, y_coordinates = create_coordinate_axes(IMAGE_SIZE,  
 PIXEL_SIZE)

    for triangle in triangles:
        projected_vertices = triangle[:, :2]
        inverse_matrix = barycentric_inverse(projected_vertices)
        normal = unit_triangle_normal(triangle)

        x_min, x_max = projected_vertices[:, 0].min(), projected_vertices[:, 0].
 max()
        y_min, y_max = projected_vertices[:, 1].min(), projected_vertices[:, 1].
 max()

        x_indices = np.where((x_coordinates >= x_min - PIXEL_SIZE) &  
 (x_coordinates <= x_max + PIXEL_SIZE))[0]
        y_indices = np.where((y_coordinates >= y_min - PIXEL_SIZE) &  
 (y_coordinates <= y_max + PIXEL_SIZE))[0]
        if x_indices.size == 0 or y_indices.size == 0:
            continue

        for py in y_indices:
            for px in x_indices:
                point_xy = np.array([x_coordinates[px], y_coordinates[py]])
                weights = barycentric_weights(point_xy, inverse_matrix)
                if np.min(weights) < -1e-9:
                    continue
                point = weights @ triangle
                depth = point[2]
                if depth <= z_buffer[py, px]:
                    continue
                if eye_position is None:
                    intensity = shader(point, normal, light_position)
                else:
                    intensity = shader(point, normal, light_position,  
 eye_position, shininess)
                image[py, px] = np.clip(intensity, 0.0, 255.0)
                z_buffer[py, px] = depth
                mask[py, px] = True

    return image, mask

```

```

def summarize_image(label: str, image: np.ndarray, mask: np.ndarray) -> None:
    """Wypisuje podstawowe statystyki wygenerowanego obrazu."""
    visible = image[mask]
    print(
        f'{label}: min = {visible.min():.2f}, max = {visible.max():.2f},  

        ↳średnia = {visible.mean():.2f}, '  

        f'liczba pikseli obiektu = {visible.size}'
    )

lambert_single_results = {}
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for axis, (label, light_position) in zip(axes, LIGHT_POSITIONS.items()):
    image, mask = render_scene([TRIANGLE_1], lambert_intensity, light_position)
    lambert_single_results[label] = (image, mask)
    axis.imshow(image, cmap='gray', vmin=0, vmax=255)
    axis.set_title(f'Model Lamberta, {label}')
    axis.axis('off')

plt.tight_layout()
plt.show()

for label, (image, mask) in lambert_single_results.items():
    summarize_image(label, image, mask)

```

Model Lamberta, Światło (a)



Model Lamberta, Światło (b)



Światło (a): min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli obiektu = 7200

Światło (b): min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli obiektu = 7200

2.1.1 Wnioski

W modelu Lamberta jasność zależy od kąta między normalną a kierunkiem do źródła światła, dlatego położenie źródła wyraźnie zmienia rozkład tonów na powierzchni trójkąta. Zastosowane tłumienie odległości powoduje dodatkowe osłabienie fragmentów dalszych od źródła, mimo że normalna całego trójkąta pozostaje stała.

2.2 Zadanie 2.

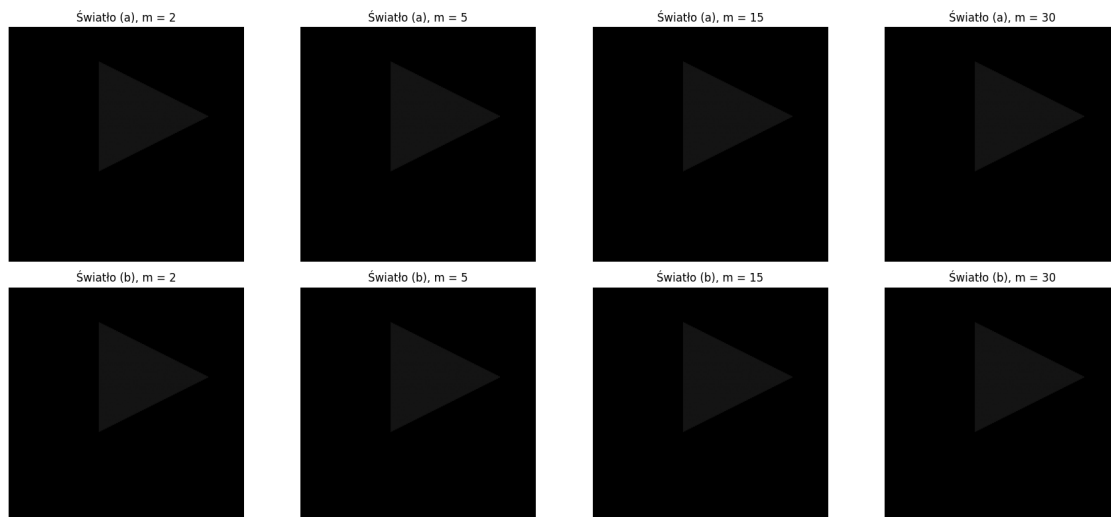
Dla tego samego trójkąta stosuję model Phong'a i sprawdzam, jak wykładnik m wpływa na koncentrację składowej lustrzanej. Używam tego samego rzutowania równoległego na OXY, ale składowa specularna dodatkowo zależy od położenia oka obserwatora [5, 5, 3].

```
[3]: phong_single_results = {}
fig, axes = plt.subplots(len(LIGHT_POSITIONS), len(PHONG_EXPONENTS),
    figsize=(18, 8))

for row, (light_label, light_position) in enumerate(LIGHT_POSITIONS.items()):
    for col, shininess in enumerate(PHONG_EXPONENTS):
        axis = axes[row, col]
        image, mask = render_scene([TRIANGLE_1], phong_intensity,
    light_position, OBSERVER_EYE, shininess)
        phong_single_results[(light_label, shininess)] = (image, mask)
        axis.imshow(image, cmap='gray', vmin=0, vmax=255)
        axis.set_title(f'{light_label}, m = {shininess}')
        axis.axis('off')

plt.tight_layout()
plt.show()

for (light_label, shininess), (image, mask) in phong_single_results.items():
    summarize_image(f'{light_label}, m = {shininess}', image, mask)
```



```

Światło (a), m = 2: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (a), m = 5: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (a), m = 15: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (a), m = 30: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (b), m = 2: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (b), m = 5: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (b), m = 15: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200
Światło (b), m = 30: min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli
obiekту = 7200

```

2.2.1 Wnioski

Składowa lustrzana w modelu Phong'a staje się coraz bardziej skupiona wraz ze wzrostem wykładnika m , więc dla dużych wartości pojawia się mały, ale znacznie jaśniejszy refleks. Położenie oka obserwatora ma bezpośredni wpływ na miejsce i siłę odbicia, dlatego obrazy Phong'a różnią się od wersji Lambertowskiej nawet przy tej samej geometrii i tym samym źródle światła.

2.3 Zadanie 3.

Powtarzam oba modele oświetlenia dla sceny złożonej z dwóch trójkątów przylegających wspólną krawędzią. Używam bufora głębokości, dzięki czemu w razie nakładania projekcji zachowywana jest poprawna widoczność fragmentów o większej współrzędnej z .

```

[4]: TWO_TRIANGLE_SCENE = [TRIANGLE_1, TRIANGLE_2]

lambert_double_results = {}
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for axis, (label, light_position) in zip(axes, LIGHT_POSITIONS.items()):
    image, mask = render_scene(TWO_TRIANGLE_SCENE, lambert_intensity,
    ↪light_position)
    lambert_double_results[label] = (image, mask)
    axis.imshow(image, cmap='gray', vmin=0, vmax=255)
    axis.set_title(f'Dwa trójkąty, Lambert, {label}')
    axis.axis('off')

plt.tight_layout()
plt.show()

```

```

for label, (image, mask) in lambert_double_results.items():
    summarize_image(f'Dwa trójkąty, {label}', image, mask)

phong_double_results = {}
fig, axes = plt.subplots(len(LIGHT_POSITIONS), len(PHONG_EXPONENTS),
    figsize=(18, 8))

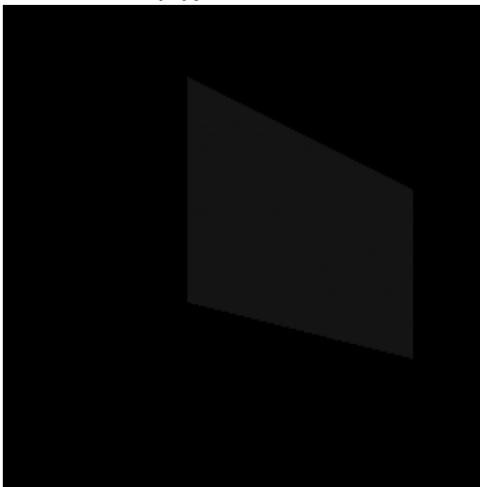
for row, (light_label, light_position) in enumerate(LIGHT_POSITIONS.items()):
    for col, shininess in enumerate(PHONG_EXPONENTS):
        axis = axes[row, col]
        image, mask = render_scene(TWO_TRIANGLE_SCENE, phong_intensity,
    light_position, OBSERVER_EYE, shininess)
        phong_double_results[(light_label, shininess)] = (image, mask)
        axis.imshow(image, cmap='gray', vmin=0, vmax=255)
        axis.set_title(f'Dwa trójkąty, {light_label}, m = {shininess}')
        axis.axis('off')

plt.tight_layout()
plt.show()

for (light_label, shininess), (image, mask) in phong_double_results.items():
    summarize_image(f'Dwa trójkąty, {light_label}, m = {shininess}', image,
    mask)

```

Dwa trójkąty, Lambert, Światło (a)

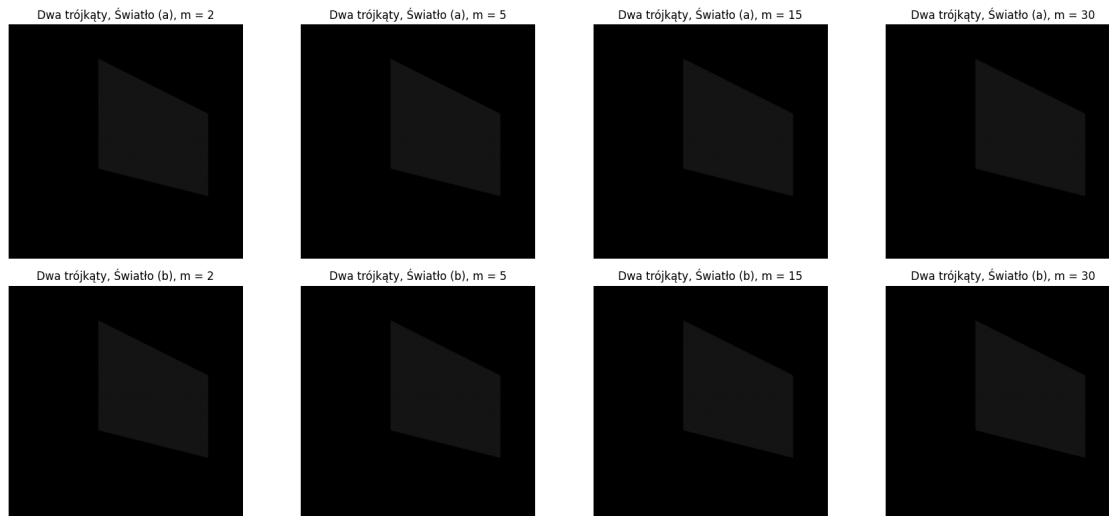


Dwa trójkąty, Lambert, Światło (b)



Dwa trójkąty, Światło (a): min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli obiektu = 12600

Dwa trójkąty, Światło (b): min = 20.00, max = 20.00, średnia = 20.00, liczba pikseli obiektu = 12600



Dwa trójkąty, Światło (a), $m = 2$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (a), $m = 5$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (a), $m = 15$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (a), $m = 30$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (b), $m = 2$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (b), $m = 5$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (b), $m = 15$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600
 Dwa trójkąty, Światło (b), $m = 30$: min = 20.00, max = 20.00, średnia = 20.00,
 liczba pikseli obiektu = 12600

2.3.1 Wnioski

Dla dwóch trójkątów różnice orientacji normalnych dają wyraźną zmianę jasności na wspólnej scenie, mimo że obie ściany są oświetlane tym samym źródłem. Model Phong dodatkowo wzmacnia lokalne refleksy na tych fragmentach powierzchni, dla których kierunek odbicia zbliża się do kierunku obserwatora, a bufor głębokości gwarantuje poprawne odwzorowanie widoczności.